ELI — Project Overview

ELI v2.5.12

October 2026

Contents

ELI — Full Project Breakdown & Assessment **1**
1. What ELI is . . . 1
2. Scale and shape . . . 2
3. Architecture, layer by layer . . . 2
4. What is genuinely strong . . . 3
5. Where it is weak (measured) . . . 3
6. Verdict . . . 4
7. Highest-leverage work . . . 4
8. Honest limits that are by design . . . 4

ELI — Full Project Breakdown & Assessment

> **Updated for v2.4.84 (September 2026).** Every CHAT mode runs the gradient orchestrator, retrieval is shared in `eli/memory/retrieval.py`, memory is governed by a storage policy (`eli/memory/policy.py`), and the S01–S12 pipeline is traced canonically. Installers are CI-launch-tested and published on GitHub Releases.

A grounded read of the whole project: what ELI is, its scale and shape, the architecture layer by layer, where it is strong, where it is weak (with measured numbers), and what would help most. Companion to `orchestration_and_agents.md` (agent and bus detail).

> Method note: the numbers below were computed from the tree on 2026-09-25 (AST parse of every file under `eli/`, `pytest --collect-only`, the capability manifest). Prose claims were checked against the code they describe.

---

1. What ELI is

A **local-first, model-agnostic personal AI**: no cloud, no telemetry, offline by default and hard-gated at the socket boundary. Internet access is an owner-controlled, monitored opt-in; one-time opt-in model downloads are the only network event unless the owner enables the Internet toggle. It bundles GGUF inference (llama.cpp), a PySide6 desktop GUI, persistent SQLite + FTS5 + FAISS memory, a knowledge graph, a 12-stage cognitive pipeline with a parallel 15-agent bus, a deterministic grounding and evidence layer that fights confabulation, local vision (Qwen2.5-VL hot-swap and Moondream), TTS and STT, OS control, a plugin system, a proactive daemon, and a LoRA self-training loop.

It also has a second front end: a self-hosted **FastAPI web app and dashboard PWA** (`api/server.py`): chat, a live telemetry dashboard, ELI's own MQTT smart-home (rooms, scenes, automations), multi-user roles (admin, member, viewer), a hash-chained audit trail, shared research corpora, browser voice and the monitored Internet toggle. It is launchable in-process from the desktop GUI.

2. Scale and shape

194,736 lines across 429 Python files in `eli/`, plus `api/server.py` (2,292 lines with an embedded dashboard) and **544 test files** under `tests/`.

Subsystem	Lines	Files	Role
`runtime/`	35.4k	95	grounding, evidence, introspection, surfaces, daemons
`gui/`	27.4k	27	PySide6 desktop app
`execution/`	26.3k	13	router and executor (action dispatch)
`cognition/`	20.8k	47	agent bus, orchestrator, inference, persona, modes
`kernel/`	17.5k	8	the engine (pipeline driver)
`core/`	11.2k	30	paths, settings, hardware profile
`perception/`	10.2k	23	vision, STT, TTS, OS control
`memory/`	8.8k	11	SQLite, FTS5, FAISS, knowledge graph, storage policy
`tools/`	7.6k	29	image engine, news, registry
`plugins/`	5.9k	34	plugin manager, marketplace client
`learning/`	4.3k	14	LoRA self-training
`planning/`	4.0k	19	proactive daemon, habits, queues
`integrations/`	3.3k	18	Ollama, MCP, cross-platform media
`coding/`	2.1k	12	the coding agent
`setup/`, `utils/`, `world/`, `system/`, `contracts/`, `onboarding/`, `cli/`	8.5k	52	installer, helpers, world model, portable control

Five files carry about a third of the codebase: `engine.py` (16.0k), `executor_enhanced.py` (15.8k), `gui/eli_pro_audio_gui_v2_0.py` (13.1k), `router_enhanced.py` (8.3k) and `gui/labs_tab.py` (5.7k). Next tier: `memory.py` (5.7k), `agent_bus.py` (3.5k), `deterministic_grounding_gate.py` (3.4k).

3. Architecture, layer by layer

- **Boot and hardware**: `core/hardware_profile.py` and `core/startup_hardware_optimizer.py` fit context, GPU layers and batch to whatever model and GPU are present. One fit calculation is shared by the loader and the startup dialog; the fit priority (balanced, max GPU, max context) is a setting.

- **Routing**: `execution/router_enhanced.py` is regex-first with an LLM-intent fallback and an explicit priority pipeline, resolving to one of **230 manifest capabilities** (189 routable, 207 in the executor's supported list, 213 in either). The full reference with activation phrases is `capabilities_and_actions.md`.
- **Orchestration**: `kernel/engine.py` runs the gradient orchestrator for all CHAT modes (Quick is light, Expert is deep); `dispatch_specialists()` composes the 15-agent `cognition/agent_bus.py` after shared retrieval. The bus is a fallback if orchestration fails. See `orchestration_and_agents.md`.
- **Inference**: `cognition/gguf_inference.py` resolves the model path from environment and settings (no baked model), serialises calls behind a lock, supports a live runtime override, and hot-swaps with vision models. Model family and chat template come from GGUF metadata (`model_identity.py`).
- **Memory**: `memory/retrieval.py` owns turn retrieval (a question naming a period is filtered by date first); `memory/memory.py` is the SQLite, FTS5, FAISS and knowledge graph foundation; `memory/policy.py` decides what is stored, merged, decayed and archived (`memory.md`).
- **Grounding and evidence**: `runtime/deterministic_grounding_gate.py`, `evidence_ledger.py`, `evidence_arbitration.py`, `control_contracts.py`, `grounded_remediation.py` and `cognition/output_governor.py`: a layered, deterministic anti-confabulation system around the probabilistic model.
- **Security**: `runtime/security.py` `SecurityManager` with a fail-closed shell gate (`ELI_ALLOWED_CMDS` or the Full Control toggle; unset means blocked), path allow-roots (`ELI_ALLOW_ROOTS`), an app allowlist, a SHA-256 custom-agent trust registry (`cognition/agent_trust.py`, stored in `config/trusted_agents.json`), SQL identifier validation in `memory/memory.py`, and prompt-injection screening in the engine.
- **Self-model**: `cognition/reasoning_modes.py`: `quick`, `fast` and `balanced` all fast-path to quick, plus four private modes (`chain_of_thought`, `self_consistency`, `tree_of_thoughts`, `constitutional_ai`). Persona overlay, `runtime/self_improvement.py` and LoRA self-training (`learning/`).

4. What is genuinely strong

1. **The grounding and evidence layer.** Deterministic evidence gating around a probabilistic model, developed to a high degree.
2. **Local and model-agnostic.** No call-home, hardware-adaptive, swappable models.
3. **Fail-closed security.** Shell blocked by default, hash-gated custom agents, path and app allowlists.
4. **Breadth, integrated.** Vision, voice, OS control, memory, knowledge graph, plugins and self-training in one pipeline.
5. **A large safety net.** 12,496 tests are collected; the structural tests catch broken imports, stale action lists and unpinned routes.

5. Where it is weak (measured)

1. **Swallowed errors.** 4,392 handlers catch `Exception` (most log at debug level), 5 are bare `except:`, and 162 are a lone `pass`. A ratchet test (`tests/claims/test_no_silent_swallow.py`) stops the silent count rising.
2. **Large files.** The executor is an action ladder and the engine a long orchestration method; the regression surface is high and unit-testing is hard.
3. **Overlap in `runtime/`.** Many `personal_memory_*`, `*_surface` and `*_response` modules do neighbouring grounding work.
4. **Single-file GUI.** `eli_pro_audio_gui_v2_0.py` (13.1k lines) and `labs_tab.py` (5.7k).
5. **Test-suite state.** See `operations.md` for the current pass count; tests that touch settings and path caches are isolated from one another by fixtures in `tests/conftest.py`.
6. **Network containment is Python-level.** There is no eBPF, seccomp, landlock, namespace or firewall integration. The socket guard covers in-process code only; anything that spawns a subprocess (MCP servers, `pip`, generated scripts) is outside it.

6. Verdict

In ambition and in specific subsystems, ELI is ahead of most open local-assistant projects: the grounding layer, the fully local model-agnostic design and the integrated multimodal agent. **In engineering discipline it is not finished:** the swallowed exceptions, the large files and the overlapping modules separate an extraordinarily ambitious solo project from software others can build on. The gap is subtraction and observability, not more features.

7. Highest-leverage work

1. **Make swallowed errors observable.** Replace blanket `except Exception: pass` with scoped types and a single structured error log, keeping graceful degradation.
2. **Split the two largest files** along their natural seams (executor action groups behind the dispatch table; engine stages into modules).
3. **Fold overlapping `runtime/` surfaces** into a handful of well-named modules.
4. **Split the GUI monoliths** the way the engine is meant to be split.

8. Honest limits that are by design

- Refusing to claim more than was checked: a scanner engine that could not run never counts as a pass and the verdict says coverage was partial; an unsigned plugin is reported as unverified; ELI never says a community plugin is safe; the training tab says the live model has not changed until the adapter is merged; the MCP screens state that the network guard cannot contain a child process.
- Plugins are declared, verified, scanned and permission-gated; custom agents carry a specification and a provenance-carrying trust chain; the LoRA trainer has a human review gate.